

DeepSeek-Coder-1.3B

导论

DeepSeek-Coder 是 DeepSeek 推出的开源代码大语言模型系列，主要面向代码生成、代码补全、代码理解与程序修复等任务。该系列通过大规模代码数据、项目级文件组织、长上下文训练和 Fill-In-the-Middle (FIM) 任务，增强了模型对函数调用、模块依赖和跨文件代码结构的理解能力。

本文介绍的 **DeepSeek-Coder-1.3B** 是该系列中规模较小的模型。它采用 Decoder-only Transformer 架构，共包含 24 个 Decoder Block，隐藏维度为 2048，具有 16 个注意力头，SwiGLU 前馈网络的中间维度为 5504，词表大小为 32256，并支持最长 16K Token 的上下文。模型内部使用 RMSNorm、RoPE、多头因果自注意力和残差连接，其整体结构与早期 LLaMA 类模型相近。

在推理过程中，输入的自然语言或源代码首先经过 Byte-level BPE Tokenizer，被转换为 Token 编号；随后通过 Token Embedding 映射为向量，并依次经过 24 个 Decoder Block。最终隐藏状态经过 RMSNorm 和 LM Head，被映射为整个词表上的 Logits，再通过 Softmax 和解码策略选择下一个 Token。模型不断重复这一过程，直到生成结束标记，从而得到完整代码或文本。

DeepSeek-Coder-1.3B 的主要特点并不是对 Transformer 结构进行大幅修改，而是通过高质量代码数据、仓库级依赖组织、FIM 训练和长上下文扩展提升代码能力。由于模型规模较小、结构清晰且权重公开，它也适合用于本地部署、模型微调以及 Transformer 架构与自回归生成过程的学习。

设：

符号	含义	DeepSeek-Coder-1.3B 中的数值
B	批量大小	示例中为 1
T_0	输入提示的 Token 数	由分词结果决定
V	词表大小	32256
d	隐藏维度	2048
L	解码器层数	24
H	注意力头数	16
d_h	每个注意力头的维度	128
d_{ff}	MLP 中间维度	5504
S	生成的输出 Token 数	由生成结果决定

每个注意力头的维度为 $d_h = \frac{d}{H} = \frac{2048}{16} = 128$

模型生成完整回答的条件概率为 $P(y_1, y_2, \dots, y_S | x_1, x_2, \dots, x_{T_0})$

根据概率链式法则，可以分解为 $P(y_1, \dots, y_S | x_{1:T_0}) = \prod_{s=1}^S P(y_s | x_{1:T_0}, y_{1:s-1})$

1. 模块 A：源代码或自然语言输入

用户输入：请用一句话解释什么是递归。

在程序中，它首先是一个 Unicode 字符串，而不是模型能够直接计算的数值。

Unicode 字符串是什么？

Unicode 为世界上几乎所有文字、符号和表情分配了统一的编号，因此 Unicode 字符串可以同时包含中文、英文、数学符号、日文和 Emoji 等内容

例如：Hello，世界！😊

这里的每个字符都有对应的 Unicode 码点：

A → U+0041

中 → U+4E2D

😊 → U+1F60A

需要区分两个概念：

Unicode 字符串：表示“有哪些字符”。

字节序列：表示这些字符在计算机存储或网络传输时具体使用哪些字节。

Unicode 字符串需要通过某种**字符编码**转换为字节，常见编码包括 UTF-8、UTF-16 和 UTF-32。

例如字符“中”使用 UTF-8 编码后对应三个字节：中 → E4 B8 AD

因此，Unicode 本身不是 UTF-8。Unicode 是字符编号标准，UTF-8 是 Unicode 的一种具体编码方式。

模型只能对整数和浮点数张量执行矩阵运算，因此必须经历

文字 → 字节 → 子词 Token → Token 编号 → 向量

Instruct 模型的额外模板

对于 Instruct 模型，用户看到的句子通常会被组织成类似：

<句首 Token>

系统提示

Instruction:

请用一句话解释什么是递归。

Response:

因此，真正进入模型的上下文往往不只有用户输入的一句话，还包含句首特殊 Token；系统提示；Instruction 标记；Response 标记。

官方 Instruct 分词器配置 (https://huggingface.co/deepseek-ai/deepseek-coder-1.3b-instruct/blob/main/tokenizer_config.json) 中启用了句首 Token，并使用聊天模板构造指令格式。为了集中解释模型数学结构，后文统一把最终送入分词器的完整字符串记为 X_{text}

2. 模块 B: Byte-level BPE Tokenizer

为什么不能按照汉字或单词直接处理？

模型词表不可能包含所有可能的中文句子；英文单词；变量名称；文件路径；代码标识符；Unicode 字符组合。

因此 DeepSeek Coder 使用字节级 BPE，将文本划分为有限词表中的子词单元。DeepSeek Coder 论文描述其使用约 32K 规模的字节级 BPE 词表，公开配置中的实际词表大小为 32256。

UTF-8 字节化

首先将文本编码成 UTF-8 字节序列： $X_{\text{text}} \rightarrow (b_1, b_2, \dots, b_M)$

其中 $b_i \in \{0, 1, \dots, 255\}$ ，一个中文字符通常会占用多个 UTF-8 字节。例如，“递”不会直接作为一个数学变量进入模型，而会先被编码为若干字节。

BPE 合并

初始状态下，可以把每个字节视为一个基本符号： $(s_1, s_2, \dots, s_M)$

BPE 词表包含一组预先学习到的合并规则，例如示意性地：

符号 A + 符号 B $\rightarrow$ 新符号 AB

符号 AB + 符号 C $\rightarrow$ 新符号 ABC

分词器按照合并优先级不断寻找可合并的相邻符号，直到不能继续合并。

因此一句话可能被切分成：

[句首 Token, 请, 用, 一句话, 解释, 什么, 是, 递归, 。, 回答标记]

这只是示意切分，实际切分结果由 DeepSeek 的具体词表和合并规则决定，不能简单地认为一个汉字必然对应一个 Token。

设最终得到 T_0 个 Token： $\tau_1, \tau_2, \dots, \tau_{T_0}$

分词器词表映射为 $\text{id}: \tau_i \rightarrow x_i$ ，其中 $x_i \in \{0, 1, \dots, 32255\}$

于是文本变为整数序列 $(x_1, x_2, \dots, x_{T_0})$

3. 模块 C: Token 编号 $B \times T$

对于一个批量中的输入，整数张量记为 $X_{\text{id}} \in \mathbb{N}^{B \times T_0}$

在当前示例中 $B = 1$ ，因此 $X_{\text{id}} \in \mathbb{N}^{1 \times T_0}$

形式上可以写成 $X_{\text{id}} d = [x_1, x_2, x_3, \dots, x_{T_0}]$

每个 x_i 只是一个词表下标。例如：

“递归” $\rightarrow$ 某个 Token 编号

“函数” $\rightarrow$ 某个 Token 编号

“。” $\rightarrow$ 某个 Token 编号

编号的大小没有数值语义。编号 30000 并不比编号 1000 表示“更大的概念”，它们只是查表地址。

4. 模块 D: Token Embedding

整数编号不能直接参加注意力计算，因此要通过嵌入矩阵变成向量。

嵌入矩阵

DeepSeek-Coder-1.3B 的嵌入矩阵为: $E \in \mathbb{R}^{V \times d}$

代入数值 $E \in \mathbb{R}^{32256 \times 2048}$

矩阵的第 i 行表示 Token 编号 i 对应的可学习向量 $E_i \in \mathbb{R}^{2048}$

查表过程

对于第 t 个 Token 编号 x_t , 嵌入为 $h_t^{(0)} = E_{x_t}$, 将全部 Token 排列起来 $H^{(0)} = \begin{bmatrix} h_1^{(0)} \\ h_2^{(0)} \\ \vdots \\ h_{T_0}^{(0)} \end{bmatrix}$,

这里上标 (0) 表示尚未进入第一个解码器层。

嵌入向量

初始嵌入向量只取决于 Token 身份 $h_t^{(0)} = E_{x_t}$, 同一个 Token 出现在不同句子中, 初始嵌入相同。例如, “函数”出现在:

定义一个函数

这个函数调用自身

初始嵌入是相同的。它的上下文含义要经过自注意力计算后才会发生变化。

5. 模块 E: 隐藏状态 $B \times T \times 2048$

所谓隐藏状态, 就是模型内部保存的 Token 表示。

初始隐藏状态: $H^{(0)} \in \mathbb{R}^{B \times T_0 \times 2048}$, 经过第 1 个解码器层后: $H^{(l)} \in \mathbb{R}^{B \times T_0 \times 2048}$

虽然经过每一层后张量形状不变, 但数值和含义会变化:

初始嵌入

→ 局部 Token 特征

→ 语法结构

→ 长距离依赖

→ 与回答任务相关的上下文表示

不能把某一层简单地固定解释为“语法层”或“语义层”, 但随着层数增加, 每个位置的向量会融合更多上下文信息。

6. 模块 F: 构造因果掩码和位置编号

这个模块实际上包含两种不同的信息: 1. 因果掩码决定“允许看哪些 Token”; 2. 位置编号决定“每个 Token 位于什么位置”。

因果掩码

Decoder-only 模型必须保证位置 i 不能读取未来位置 $j > i$ 。

定义因果掩码: $M_{ij} = \begin{cases} 0, & j \leq i \\ -\infty, & j > i \end{cases}$

当 $T_0 = 4$ 时: $M = \begin{pmatrix} 0 & -\infty & -\infty & -\infty \\ 0 & 0 & -\infty & -\infty \\ 0 & 0 & -\infty & -\infty \\ 0 & 0 & 0 & 0 \end{pmatrix}$, 含义是第一个 Token 只能看自己; 第二

个 Token 可以看前两个 Token; 第三个 Token 可以看前三个 Token; 最后一个 Token 可以看整个已有输入。

掩码最终会扩展为 $M \in \mathbb{R}^{B \times 1 \times T_q \times T_k}$, 其中 T_q 是查询长度; T_k 是键值长度。

初次处理完整提示词时 $T_q = T_k = T_0$

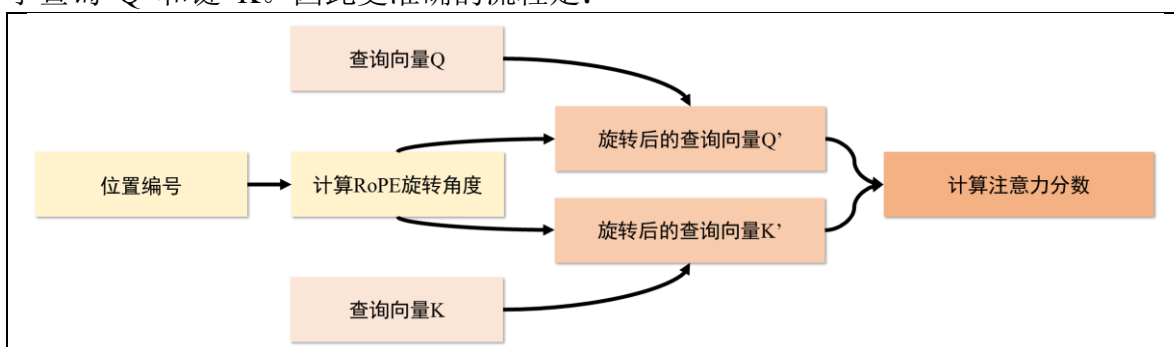
位置编号

输入序列的位置编号为 $P = [0, 1, 2, \dots, T_0 - 1]$, 即 $P \in \mathbb{N}^{1 \times T_0}$

如果已经缓存了长度为 T_{past} 的历史 Token, 新 Token 的位置从 T_{past} 开始, 而不是重新从 0 开始。源码中位置编号是按照历史缓存长度构造。

位置编号不会直接加到 Token 嵌入上

经典 Transformer 常写成 Token 嵌入+位置嵌入, 但 LLaMA 类模型没有把一个可学习位置向量直接加到 $H^{(0)}$ 上。位置编号会在每一层自注意力中, 通过 RoPE 作用于查询 Q 和键 K。因此更准确的流程是:

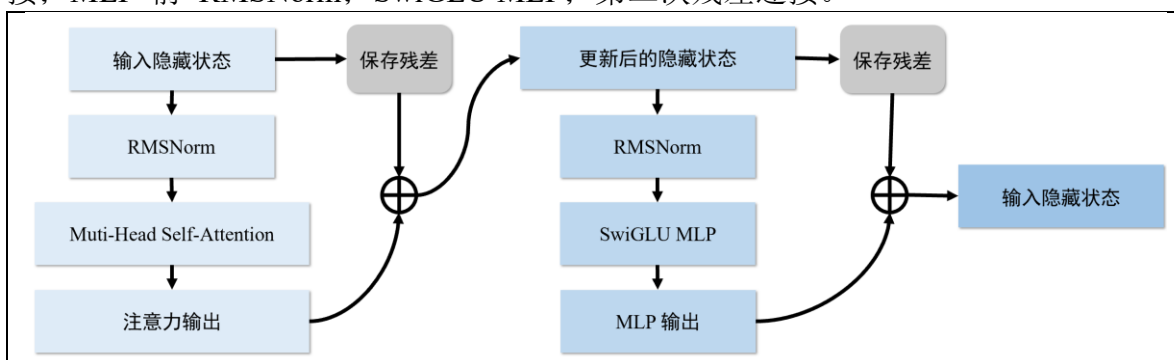


7. 模块 G—J: 24 个 Decoder Block

模型依次执行 $H^{(1)} = F_1(H^{(0)})$, $H^{(2)} = F_2(H^{(1)})$, $H^{(24)} = F_{24}(H^{(23)})$

虽然 24 个层的结构相同, 但参数不共享: $F_1 \neq F_2 \neq \dots \neq F_{24}$

每个 Decoder Block 都包含注意力前 RMSNorm; 多头自注意力; 第一次残差连接; MLP 前 RMSNorm; SwiGLU MLP; 第二次残差连接。



RMSNorm

设第 1 层输入中某个位置的向量为 $x \in \mathbb{R}^{2048}$

RMSNorm 首先计算均方值 $MS(x) = \frac{1}{2048} \sum_{i=1}^{2048} x_i^2$

再计算均方根 $RMS(x) = \sqrt{\frac{1}{2048} \sum_{i=1}^{2048} x_i^2 + \epsilon}$ ，其中 $\epsilon = 10^{-6}$

最终 $RMSNorm(x) = g \odot \frac{x}{\sqrt{\frac{1}{2048} \sum_{i=1}^{2048} x_i^2 + \epsilon}}$ ，其中 $g \in \mathbb{R}^{2048}$ 是可学习缩放向量；

$\odot$ 表示逐元素乘法；输出维度仍然是 2048。RMSNorm 不会减去均值，这一点不同于 LayerNorm。

计算 Q、K、V

设归一化后的张量为 $\bar{H}^{(l)} = RMSNorm(H^{(l)})$ ，其形状仍为 $\bar{H}^{(l)} \in \mathbb{R}^{B \times T \times 2048}$

对归一化结果执行三个线性变换： $Q = \bar{H}^{(l)} W_Q^T$ ； $K = \bar{H}^{(l)} W_K^T$ ； $V = \bar{H}^{(l)} W_V^T$

对于 DeepSeek-Coder-1.3B： $W_Q, W_K, W_V \in \mathbb{R}^{2048 \times 2048}$ ，所以 $Q, K, V \in \mathbb{R}^{B \times T \times 2048}$

然后拆分为 16 个头：

$$Q \longrightarrow \mathbb{R}^{B \times 16 \times T \times 128}, K \longrightarrow \mathbb{R}^{B \times 16 \times T \times 128}, V \longrightarrow \mathbb{R}^{B \times 16 \times T \times 128}$$

因为 $num_key_value_heads = num_attention_heads = 16$

所以该模型使用标准多头注意力 MHA，而不是 GQA。相关投影层和张量重排过程可以在对应版本的 LlamaAttention 中直接看到。

RoPE 位置旋转

每个注意力头的维度为 $d_h = 128$ ，因此可以组成 64 个二维坐标对。第 r 个坐标

对的基础频率可以写为： $\omega_r = \frac{1}{100000^{2r/128}}$ ，其中 $r = 0, 1, \dots, 63$ ，DeepSeek-Coder-

1.3B 的配置使用 $rope_theta = 100000$ ，并启用线性缩放 $factor = 4$

官方配置和 v4.34.1 源码表明，线性缩放通过把位置值除以缩放因子实现，即有效位置使用 $p/4$ 。

位置 p 、频率对 r 的旋转角度为 $\theta_{p,r} = \frac{p}{4} \omega_r$

二维旋转矩阵为 $R(\theta_{p,r}) = \begin{bmatrix} \cos \theta_{p,r} & -\sin \theta_{p,r} \\ \sin \theta_{p,r} & \cos \theta_{p,r} \end{bmatrix}$

对于查询向量中的一个二维分量 $\begin{bmatrix} q_{2r}' \\ q_{2r+1}' \end{bmatrix} = R(\theta_{p,r}) \begin{bmatrix} q_{2r} \\ q_{2r+1} \end{bmatrix}$

键向量同样旋转 $\begin{bmatrix} k_{2r}' \\ k_{2r+1}' \end{bmatrix} = R(\theta_{p,r}) \begin{bmatrix} k_{2r} \\ k_{2r+1} \end{bmatrix}$

值向量 V 不执行 RoPE。

源码等价地实现为：

$$Q' = Q \odot \cos\Theta + \text{rotate}(Q) \odot \sin\Theta; \quad K' = K \odot \cos\Theta + \text{rotate}(K) \odot \sin\Theta$$

对应源码中的 `apply_rotary_pos_emb`。

缩放点积注意力

对于第 h 个注意力头 $Q_h', K_h', V_h \in \mathbb{R}^{B \times T \times 128}$

查询和键相乘 $S_h = \frac{Q_h' K_h'^T}{\sqrt{128}}$ ，其形状为 $S_h \in \mathbb{R}^{B \times T \times T}$ ，其中 $S_{h,i,j}$ 表示第 i 个

Token 对第 j 个 Token 的相关性分数，除以 $\sqrt{128}$ 是为了避免内积随维度增大而数值过大。

$$S_h' = S_h + M, \text{ 当 } j > i \text{ 时 } M_{ij} = -\infty, \text{ 因此 } S_{h,i,j}' = -\infty$$

$$\text{注意力权重为 } A_{h,i,j} = \frac{\exp(S_{h,i,j}')}{\sum_{r=1}^T \exp(S_{h,i,r}')}, \text{ 未来位置对应 } \exp(-\infty) = 0, \text{ 所以未来位}$$

置的注意力权重为零。

源码会把注意力分数提升到 `float32` 执行 `Softmax`，再转换回查询张量的数据类型，以改善数值稳定性。

$$O_h = A_h V_h, \text{ 形状为 } O_h \in \mathbb{R}^{B \times T \times 128}, \text{ 对于位置 } i: o_{h,i} = \sum_{j=1}^i A_{h,i,j} v_{h,j}, \text{ 表示当前位}$$

置从自己和前面的所有位置中汇总信息。

合并注意力头

将 16 个注意力头拼接: $O = \text{Concat}(O_1, O_2, \dots, O_{16})$ ，得到 $O \in \mathbb{R}^{B \times T \times 2048}$

再经过输出投影: $Y_{\text{attn}} = O W_O^T$ ，其中 $W_O \in \mathbb{R}^{2048 \times 2048}$

输出形状仍为 $Y_{\text{attn}} \in \mathbb{R}^{B \times T \times 2048}$

第一次残差连接

注意力子层采用 Pre-Norm 残差结构 $U^{(l)} = H^{(l)} + \text{Attention}(\text{RMSNorm}(H^{(l)}))$ ，也就是 $U^{(l)} = H^{(l)} + Y_{\text{attn}}$

残差连接要求两个张量形状相同 $H^{(l)}, Y_{\text{attn}} \in \mathbb{R}^{B \times T \times 2048}$ ，所以可以逐元素相加。

第二次 RMSNorm

对第一次残差结果执行 $\bar{U}^{(l)} = \text{RMSNorm}(U^{(l)})$ ，形状仍为 $\bar{U}^{(l)} \in \mathbb{R}^{B \times T \times 2048}$

这次 RMSNorm 的参数与注意力前的 RMSNorm 参数不同。每个 Decoder Block 中有两套独立的 RMSNorm 参数。

SwiGLU MLP

MLP 对每个 Token 位置独立执行。它不会直接在不同位置之间交换信息；不同位置之间的信息交换已经由自注意力完成。

Gate 投影: $G = \bar{U}^{(l)} W_{\text{gate}}^T$ ，其中 $W_{\text{gate}} \in \mathbb{R}^{5504 \times 2048}$ ，因此 $G \in \mathbb{R}^{B \times T \times 5504}$

Up 投影: $A = \bar{U}^{(l)} W_{\text{up}}^T$, 其中 $W_{\text{up}} \in \mathbb{R}^{5504 \times 2048}$, 所以 $A \in \mathbb{R}^{B \times T \times 5504}$

SiLU 定义为 $\text{SiLU}(z) = z\sigma(z)$, 其中 Sigmoid 为 $\sigma(z) = \frac{1}{1 + \exp(-z)}$, 因此

$$\text{SiLU}(z) = \frac{z}{1 + \exp(-z)}$$

$C = \text{SiLU}(G) \odot A$, 其中 $C \in \mathbb{R}^{B \times T \times 5504}$, 这一逐元素乘法表示 Gate 分支控制 Up 分支中哪些特征被保留或抑制。

Down 投影: $Y_{\text{mlp}} = C W_{\text{down}}^T$, 其中 $W_{\text{down}} \in \mathbb{R}^{2048 \times 5504}$, 因此 $Y_{\text{mlp}} \in \mathbb{R}^{B \times T \times 2048}$

合并起来 $\text{MLP}(X) = W_{\text{down}} (\text{SiLU}(W_{\text{gate}} X) \odot W_{\text{up}} X)$, 对应源码为 Gate、Up 和 Down 三个无偏置线性层。

第二次残差连接

第二条残差为 $H^{(l+1)} = U^{(l)} + \text{MLP}(\text{RMSNorm}(U^{(l)}))$, 即 $H^{(l+1)} = U^{(l)} + Y_{\text{mlp}}$

一个完整 Decoder Block 可以写为:

$$U^{(l)} = H^{(l)} + \text{Attention}(\text{RMSNorm}(H^{(l)}))$$

$$H^{(l+1)} = U^{(l)} + \text{MLP}(\text{RMSNorm}(U^{(l)}))$$

经过第 24 层后得到 $H^{(24)} \in \mathbb{R}^{B \times T \times 2048}$

8. 模块 K: Final RMSNorm

在所有 24 个 Decoder Block 之后, 再执行一次最终 RMSNorm:

$$H^{\text{final}} = \text{RMSNorm}(H^{(24)})$$

形状仍然为 $H^{\text{final}} \in \mathbb{R}^{B \times T \times 2048}$, 它的作用是统一最终隐藏状态的尺度, 再送入语言模型输出头。

9. 模块 L: 独立 LM Head: 2048 → 32256

LM Head 将每个位置的 2048 维隐藏向量转换成词表中每个 Token 的分数。

输出矩阵为 $W_{\text{lm}} \in \mathbb{R}^{32256 \times 2048}$, 对最终隐藏状态执行 $Z = H^{\text{final}} W_{\text{lm}}^T$

因此 $Z \in \mathbb{R}^{B \times T \times 32256}$, 源码中的实现是一个没有偏置的线性层:

2048 维隐藏向量 → 32256 维词表分数

DeepSeek-Coder-1.3B 配置中: `tie_word_embeddings = false`

因此输入嵌入矩阵和输出 LM Head 不共享权重: $W_{\text{lm}} \neq E$

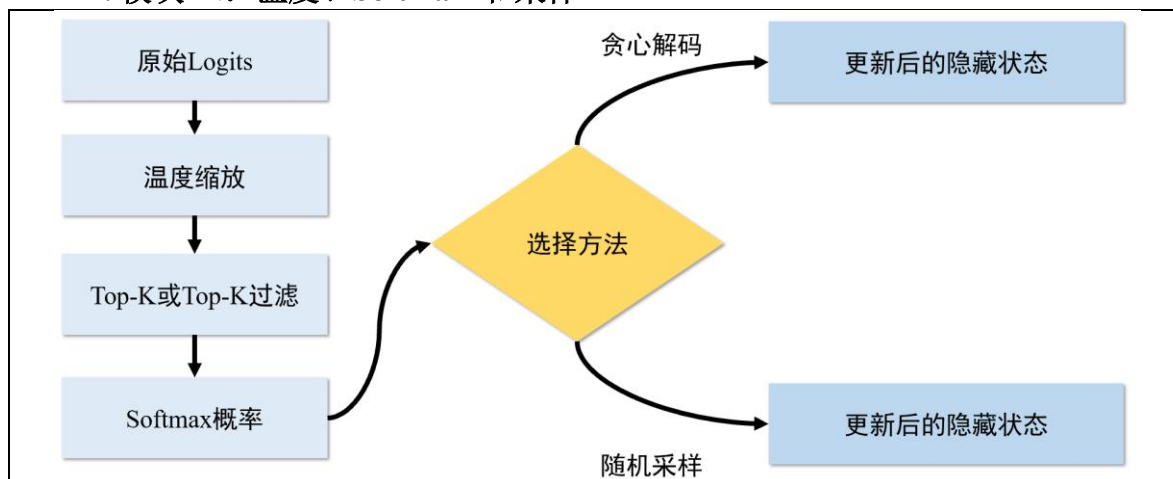
10. 模块 M: Logits $B \times T \times 32256$

Logits 是未归一化分数。对于第 t 个位置: $z_t = (z_{t,1}, z_{t,2}, \dots, z_{t,32256})$, 其中 $z_{t,i}$ 表示 Token i 作为下一个 Token 的相对倾向。

Logits 可以为正数; 可以为负数; 不要求和为 1; 不是概率。

初始提示处理完成后, 生成第一个输出 Token 只需要最后一个位置: $z_{T_0} \in \mathbb{R}^{32256}$, 因为该位置已经通过因果注意力汇总整个输入 $x_1, x_2, \dots, x_{T_0}$

11. 模块 N：温度、Softmax 和采样



温度缩放

设温度为 $\tau > 0$ ，调整后的 Logits 为 $\tilde{z}_i = \frac{z_i}{\tau}$ ，当 $\tau < 1$ ，概率分布更尖锐，更倾向高分 Token；当 $\tau > 1$ ，概率分布更平缓，随机性更强。

Softmax

Token i 的概率为 $p_i = \frac{\exp(\tilde{z}_i)}{\sum_{j=1}^{32256} \exp(\tilde{z}_j)}$ ，满足 $0 < p_i < 1$ 以及 $\sum_{i=1}^{32256} p_i = 1$

贪心解码

贪心解码选择 $y_s = \arg \max_i p_i$ ，它每次都选择当前概率最大的 Token。

随机采样表示

$y_s \sim \text{Categorical}(p_1, p_2, \dots, p_{32256})$ ，即按照 Token 概率随机抽取。

12. 模块 O：得到下一个 Token

假设模型第一次选择 $y_1 = \text{“递归”}$ ，此时还没有得到完整句子，只得到一个 Token。

当前序列从 $(x_1, \dots, x_{T_0})$ 扩展为 $(x_1, \dots, x_{T_0}, y_1)$ ，下一次预测计算 $P(y_2 | x_1, \dots, x_{T_0}, y_1)$

假设后续依次产生：

递归
是
函数
在
满足
终止条件
前
调用
自身
来
逐步
缩小
问题规模

的
方法

。

最后产生结束 Token。

13. 模块 P：是否遇到 EOS

EOS 表示 End of Sequence，即序列结束。

每次选出新 Token 后，生成控制器检查 $y_s = x_{\text{EOS}}$ ，如果 $y_s = x_{\text{EOS}}$ ，则停止生成；如果 $y_s \neq x_{\text{EOS}}$ ，则继续生成下一个 Token。

DeepSeek-Coder-1.3B Base 配置中的 EOS Token 编号为 32014；Instruct 版本还使用对话结束标记 <|EOT|> 来结束一次回答。

实际生成还可能因为其他条件结束，例如达到 max_new_tokens；达到最大上下文长度；命中自定义停止字符串；批量中的序列已经完成。

EOS 判断不是 Decoder Block 内部的神经网络层，而是外部生成控制逻辑。

14. 模块 Q：加入新 Token 并复用 KV Cache

没有 KV Cache 时，假设提示词长度为 T_0 ，生成第一个 Token 时处理 T_0 个 Token，生成第二个 Token 时，如果重新计算，则要处理 $T_0 + 1$ 个 Token。生成第三个 Token 时处理 $T_0 + 2$ 个 Token。此前所有 Token 的 K 和 V 都会被重复计算。

在因果注意力中，旧 Token 的隐藏状态不会因为后面新生成了 Token 而改变。因此旧位置在每层产生的 K_{past} 和 V_{past} 可以保存下来。

Hugging Face 的对应源码会将历史 K/V 与当前新 Token 的 K/V 在序列维度上拼接。

初始 Prefill 阶段

初次处理完整提示时 $Q, K, V \in \mathbb{R}^{B \times 16 \times T_0 \times 128}$

每一层保存 $K_{\text{cache}}^{(l)} \in \mathbb{R}^{B \times 16 \times T_0 \times 128}$ ； $V_{\text{cache}}^{(l)} \in \mathbb{R}^{B \times 16 \times T_0 \times 128}$ ，这一步称为 Prefill。

生成一个新 Token 后，下一轮只需要把这个新 Token 送入模型。新 Token 的输入形状为 $X_{\text{new}} \in \mathbb{N}^{B \times 1}$

嵌入后 $H_{\text{new}}^{(0)} \in \mathbb{R}^{B \times 1 \times 2048}$

每层新计算 $Q_{\text{new}}^{(l)} \in \mathbb{R}^{B \times 16 \times 1 \times 128}$ ； $K_{\text{new}}^{(l)} \in \mathbb{R}^{B \times 16 \times 1 \times 128}$ ； $V_{\text{new}}^{(l)} \in \mathbb{R}^{B \times 16 \times 1 \times 128}$

更新缓存 $K_{\text{cache}, \text{new}}^{(l)} = \text{Concat}(K_{\text{cache}, \text{old}}^{(l)}, K_{\text{new}}^{(l)})$ ； $V_{\text{cache}, \text{new}}^{(l)} = \text{Concat}(V_{\text{cache}, \text{old}}^{(l)}, V_{\text{new}}^{(l)})$

假设已经生成 s-1 个输出 Token，则缓存长度为： $T_s = T_0 + s - 1$

注意力分数形状从 Prefill 时的 $B \times 16 \times T_0 \times T_0$ 变为单步解码时的 $B \times 16 \times 1 \times T_s$

为什么不缓存 Query

未来生成新 Token 时，只需要新 Token 自己的 Query 去查询所有历史 Key。旧位置的 Query 不会再次使用，因此没有必要缓存。

缓存的是历史 Key；历史 Value，而不是历史 Query。

15. 模块 R：结束生成后还需要反解码

假设最终生成的输出 Token 编号为 $Y_{id} = (y_1, y_2, \dots, y_s)$ ，这些仍然是整数编号，必须经过逆词表映射 $y_s \rightarrow \tau_s$ ，得到 Token 序列 $(\tau_1, \tau_2, \dots, \tau_s)$ ，然后分词器把子词和字节重新组合为 Unicode 字符串 $\text{Decode}(Y_{id}) = Y_{\text{text}}$

最终得到：递归是函数在满足终止条件前调用自身来逐步缩小问题规模的方法。通常不会把 BOS；EOS；Padding；对话控制标记显示给用户。

16. 从输入到输出的完整链条

文本分词： $X_{\text{text}} \xrightarrow{\text{Tokenizer}} X_{id}$ ，其中 $X_{id} \in \mathbb{N}^{B \times T_0}$

Token 嵌入： $H^{(0)} = E[X_{id}]$ ，其中 $H^{(0)} \in \mathbb{R}^{B \times T_0 \times 2048}$

逐层变换：对于 $l = 0, 1, \dots, 23$ 有 $U^{(l)} = H^{(l)} + \text{Attention}_l(\text{RMSNorm}_{l,1}(H^{(l)}))$ 和 $H^{(l+1)} = U^{(l)} + \text{MLP}_l(\text{RMSNorm}_{l,2}(U^{(l)}))$

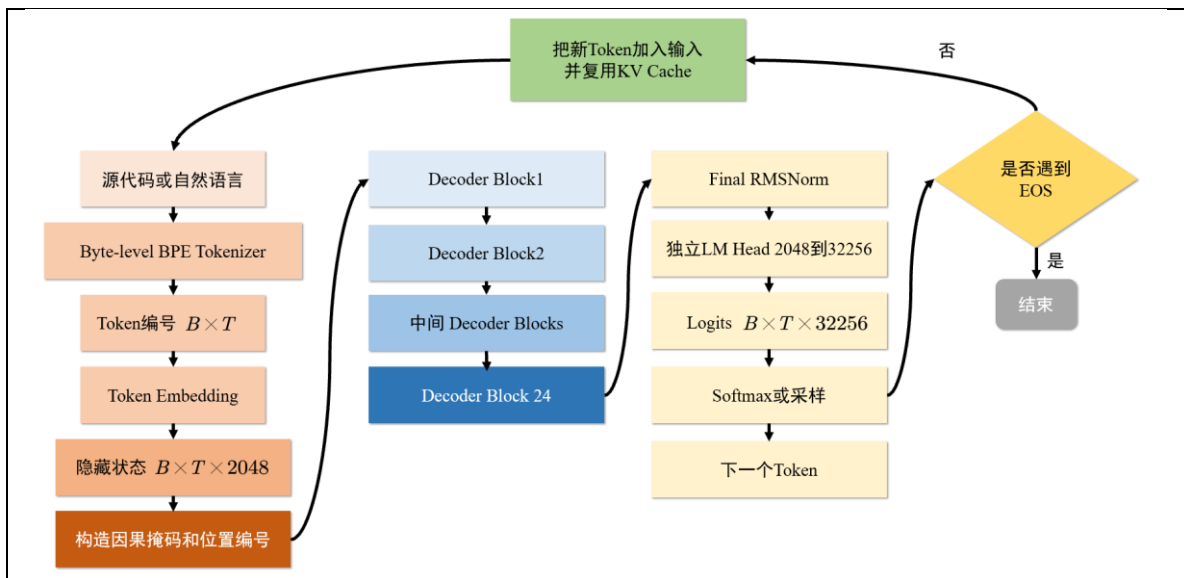
最终归一化： $H^{\text{final}} = \text{RMSNorm}_{\text{final}}(H^{(24)})$

词表 Logits： $Z = H^{\text{final}} W_{\text{lm}}^T$ ，其中 $Z \in \mathbb{R}^{B \times T_0 \times 32256}$

下一 Token 概率：取最后位置 $z = Z[:, T_0 - 1, :]$ 得 $P(y_1 = i | X) = \frac{\exp(z_i / \tau)}{\sum_{j=1}^{32256} \exp(z_j / \tau)}$

递归生成：第 s 步 $P(y_s = i | X, y_{1:s-1}) = \text{Softmax}(z^{(s)} / \tau)_i$ 选 $y_s \sim P(y_s | X, y_{1:s-1})$ ，直到 $y_s = x_{\text{EOS}}$

反解码： $Y_{\text{text}} = \text{Decode}(y_1, y_2, \dots, y_{s-1})$ ，最后一个 EOS 通常不显示。



模型是怎样学会这些概率的？

在预训练阶段，模型使用下一 Token 预测目标。

给定训练序列 $x_1, x_2, \dots, x_T$ ，输入为 $x_1, x_2, \dots, x_{T-1}$ ，标签为 $x_2, x_3, \dots, x_T$

训练损失为 $\mathcal{L} = - \sum_{t=1}^{T-1} \log P_{\theta}(x_{t+1} | x_1, \dots, x_t)$ ，其中 θ 表示模型全部参数；正确下

一个 Token 概率越大，损失越小；正确 Token 概率越小，损失越大。

例如正确下一个 Token 的预测概率为 0.8，对应损失 $-\log(0.8) \approx 0.223$ ；如果正确 Token 概率只有 0.01，对应损失 $-\log(0.01) \approx 4.605$

训练通过反向传播调整 Token Embedding；Q/K/V/O 投影；RMSNorm 参数；SwiGLU 参数；LM Head，使真实训练文本中的下一个 Token 概率逐渐增大。

DeepSeek Coder 使用大规模代码与自然语言数据进行自回归预训练，并额外使用 Fill-In-the-Middle 任务，但其最终仍可统一为下一 Token 预测。

原论文链接及部分图例：<https://arxiv.org/pdf/2401.14196v1>

Q1: Write a game snake using pygame

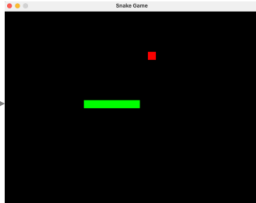


Sure, here is a simple implementation of the classic Snake game using Pygame. This code will create a window with a snake that can be controlled using the arrow keys. The snake will grow in length each time it eats the food. The game will end if the snake hits the edge of the window or its own tail.

```
...python
import pygame
import random
...
running = True
While running:
    ...
    # Draw everything
    screen.fill(BLACK)
    snake.draw()
    food.draw()
    pygame.display.flip()
    # Cap the frame rate
    clock.tick(SPEED)
# Quit Pygame
pygame.quit()
...
```

This code sets up a basic game loop where the snake moves in response to key presses, and the game ends if the snake hits the edge of the window or its own tail. The snake grows in length each time it eats the food. The game continues until the player closes the window.

Run Code



Q2: Add a scoring system in the top left corner



To add a scoring system to the top left corner of the game window, you can create a function to display the score and increment it each time the snake eats the food. Here's how you can modify the code to include a scoring system:

```
...python
...
running = True
While running:
    ...
    # Draw everything
    screen.fill(BLACK)
    snake.draw()
    food.draw()
    display_score(score) # Display the score
    pygame.display.flip()
    # Cap the frame rate
    clock.tick(SPEED)
# Quit Pygame
pygame.quit()
...
```

In this updated code, we've added a 'score' variable to keep track of the player's score, which is incremented each time the snake eats the food. The 'display_score' function is used to render the score text and display it in the top left corner of the window. The 'display_score' function is called after the snake has moved and the screen has been filled with the black color, but before the 'pygame.display.flip()' call, so that the score is drawn on top of everything else.

Run Code

